Dhrumil Bhut

Backend & AI Systems Engineer | Distributed Systems & AI Infrastructure
dhrumilbhut@gmail.com | +91 6358823190 | linkedin.com/in/dhrumilbhut | github.com/dhrumilbhut | dhrumilbhut.com

PROFESSIONAL SUMMARY

Backend and AI Engineer with 3+ years in production. Builds distributed systems and LLM infrastructure that stay correct when things fail. Stack: Node.js, TypeScript, Python. AI infra: RAG pipelines, LLM evaluation.

WORK EXPERIENCE

Backend & AI Systems Engineer 08/2023 to Present
ZURU, Ahmedabad | In-house product engineering for ZURU's consumer brands (X-Shot, Bunch O Balloons, Robo Alive).

- Own the backend for ZURU's in-house brand platforms. Built the brand asset library from scratch for 20+ brands and 20K+ media assets. Rewrote multi-filter retrieval with CTEs, cutting query time by 40% and API response time by 20%. Lambda-based preview generation cut compute costs by ~20% and load times by ~30%.
- Built the async media publishing pipeline across Instagram, TikTok and YouTube using BullMQ and Redis for job scheduling with deduplication, retries and concurrency limits, and RabbitMQ consumers to run publishing outside the request lifecycle. Added a webhook ingestion service that fans out real-time Instagram and TikTok events to internal consumers without coupling to the external API contract.
- Designed two independent rate-limiting middleware chains, one for inbound traffic and one for outbound platform quotas, with no cross-interference. Respects Meta's 1,000 req/hr cap and YouTube's 10K quota/day; the YouTube analytics sync covers 1,000+ channels without quota violations.
- Built Creator Discovery so researchers can shortlist Instagram and TikTok creators and add up to 50 to a campaign in one transaction, matching existing records, creating new ones and caching expiring marketplace photos in S3.
- Built a 10-level campaign folder hierarchy using recursive CTEs for depth and breadcrumbs, with cycle prevention and per-brand Postgres advisory locks so concurrent moves cannot corrupt the tree.
- Implemented Azure AD SSO with domain restriction and least-privilege self-provisioning, a session dashboard with remote force-logout, immediate revocation on disable or delete, and an RBAC matrix of ~130 permissions across 7 roles.
- Designed the schema and dual state machine for a multi-party licensor approval workflow: separate status lifecycles for projects and individual assets, hybrid core columns and JSONB for dynamic per-category fields, bulk asset review actions and role-based write access enforced at the brand level.

PROJECTS

BehavTest *(Behavioral Regression Testing for AI Applications)*
github.com/dhrumilbhut/behavtest | dhrumilbhut.github.io/behavtest | npmjs.com/package/behavtest

- Open source behavioral regression testing tool for LLM apps, agents and RAG pipelines. Published on npm, listed on the GitHub Marketplace, CI green on Ubuntu, macOS and Windows, versioned releases and a full documentation site.
- Core engine runs each test case through the real pipeline multiple times and compares against a baseline using statistically honest methods: Wilson confidence intervals, Fisher exact test and case-stratified permutation tests. Reports not significant when the data does not support a conclusion rather than forcing a verdict.
- Ships with RAG scorers (faithfulness at claim and answer level, retrieval with hit/recall/precision/MRR, contextRelevance per chunk), a local dashboard with run comparison and calibration views, a GitHub Action for CI and a fire-and-forget SDK that adds zero latency to the host pipeline.

ContextLens *(Self-Hosted RAG Observability Tool)* github.com/dhrumilbhut/ContextLens | contextlens.dhrumilbhut.com

- Self-hosted, Dockerized stack: Python SDK, FastAPI ingest API, Celery worker, PostgreSQL with pgvector, Redis and a dashboard, with landing page and docs site.
- Pipeline decomposes every LLM response into atomic claims, embeds claims and source chunks in a single batched API call, attributes each via cosine similarity and classifies failures as retrieval failures (chunk never fetched) or generation failures (LLM misrepresented a chunk it had). Validated on a real 14-query RAG run: initial 0.75 threshold misclassified paraphrased claims; fixed with a three-band confidence model and separate refusal detection.

PayFlow *(Payment Processing System)* github.com/dhrumilbhut/PayFlow

- Async payment processing system with Node.js, RabbitMQ, PostgreSQL and Redis. Two-layer locking via Redis SET NX and PostgreSQL FOR UPDATE ensures no payment is double-processed under concurrent worker load.
- Circuit breaker for fast-fail on gateway outages, exponential backoff retries via Dead Letter Exchange and a crash-recovery watchdog for zero stuck payments under failure. Deployed with 47 unit and integration tests covering state transitions, race conditions and webhook conflict resolution.

SKILLS

Languages	Python | JavaScript | TypeScript | SQL
Backend & Infra	Node.js | Express.js | FastAPI | AWS (Lambda, S3) | Docker | Docker Compose | CI/CD | Azure AD SSO
Data & Queues	PostgreSQL | Redis | RabbitMQ | BullMQ | Celery | Schema Design
AI Infrastructure	RAG Pipelines | pgvector | Embeddings | LLM Evaluation | OpenAI API
Testing	Jest | Unit Testing | Integration Testing | Statistical Testing

EDUCATION

B.E. Computer Science & Engineering 2020 to 2024
Darshan Institute of Engineering and Technology, Rajkot, India CGPA: 8.74